FRACTL · STAGES 2–3 · MEMBER 1

Frontend: from a Plant to a Meadow

What the frontend learned after Stage 1 — branching, random grammars, a 3D alphabet and whole scenes — how each piece was added, and how it works.

Pipeline: .fr source → lexer → parser (fractals + scene) → AST → rewriter (deterministic or **random**) → lower → turtle IR (now **13** operations) + scene layout → instance table

> IN ONE BREATH
>
> In Stage 1 the frontend understood a single fractal drawn as one line. Now it understands **branching** plants, **random** grammars that still give the same result every time, a full **3D** turtle alphabet, and **scenes** — one file that plants a tree in a field of 1,300 grass tufts. Every Stage 1 program still means exactly what it meant before.

01 What changed since Stage 1

Area	Stage 1	Now
Drawing alphabet	`F G + -` (line only)	Stage 2 `f [ ]` — pen-up and branching Stage 3 `& ^ \ / | ! '` — pitch, roll, turn around, thinner, next colour; `G` becomes a flat blade in 3D
Rules	one production per symbol	Stage 3 several weighted choices: `F -> "F+F" [0.6] | "F-F" [0.4];` plus `seed:`
File structure	exactly one `fractal`	Stage 3 several fractals + a `scene` with `place` and `scatter` (clumps, size variation)
Checks	syntax, duplicates, expansion limit	Stage 2 bracket balance at compile time · Stage 3 weights, seeds, colours, dimensions, scene references …
Known limits fixed	UTF-8 BOM rejected; brackets unchecked	both fixed in Stage 2
Turtle IR	6 operations, 3 used	13 operations, all used — new ones **appended**, old numbers unchanged
Inspection	`fr_frontend --json` for one fractal	adds max branch depth, 3D fields, and `--fractal NAME` for scene files (feeds the studio's live Stages tab)

02 Branching: `f`, `[` and `]` Stage 2

What it is. `[` saves the turtle (position and heading), `]` jumps back to it, `f` moves without drawing. With these three symbols a grammar can draw a branch, return to the stem, and draw the next branch — that is what makes plants possible.

How it was added. Because Stage 1 froze `ir.hpp` with `Move`, `Push` and `Pop` already declared, the only frontend change was three new lines in the lowering switch (`lower.hpp`). No interface changed, so Member 2's code kept compiling untouched.

```
case 'f': out = {Op::Move, d.step}; // move, pen up
case '[': out = {Op::Push, 0.0};    // save the turtle
case ']': out = {Op::Pop,  0.0};    // return to it
```

The new compile-time check. A stray `]` used to compile fine and crash the finished program. Now `checkBrackets()` in the parser checks the axiom and every rule while parsing, so the error points at the exact line and character:

```
line 5: unbalanced brackets in the rule for 'X': '[' at position 7 is never closed
```

> **WHY CHECK EACH STRING, NOT THE RESULT**
> If every rule is balanced, every expansion is balanced too (a balanced string only ever replaces a symbol). Checking the short source strings is instant and still knows line numbers; the expanded string can hold 60 million symbols and has none.

03 Random grammars that are still repeatable Stage 3

What it is. A rule may offer several productions with weights. Each time the symbol is rewritten, one is picked at random — so one grammar grows a different plant every time you change `seed:`, but the *same* seed always gives the *same* plant.

```
F -> "F[+F]F[-F]F" [0.34]
   | "F[+F]F"      [0.33]
   | "F[-F]F"      [0.33];
```

How it was added.

- **Parser** (`ruleDecl`): reads alternatives separated by `|`, each with an optional `[weight]` (default 1, must be > 0). Each alternative is bracket-checked on its own.
- **AST**: no change needed — Stage 1 already stored `Production{weight, text}` in a list. This was planned from day one.
- **Rewriter**: `chooseProduction()` multiplies a random number by the total weight and picks the first production whose running sum is larger.
- **New file** `rng.hpp`: where the random numbers come from.

The clever part: counter-based randomness

Instead of a normal random generator that you keep calling, every choice is computed from three numbers:

```
random = mix64( seed  ^  mix64( pass × P  ^  mix64( n ) ) )      // the n-th choice on that pass
```

`mix64` is the well-known SplitMix64 scrambler. Because a choice depends only on *(seed, pass, n)* — not on how many numbers were drawn before — the backend's recursive code (which works depth-first) and this rewriter (which works pass by pass) make **exactly the same choices**. Depth-first order still visits each pass's symbols from left to right, so the counters line up. The same function is written once in C++ (`rng.hpp`) and once in C (`runtime.c`), bit for bit.

> **SAY IT LIKE THIS**
> "Our randomness is a pure function of the seed, the rewrite pass and the position. That's why the unrolled compiler, which decides at compile time, and the recursive compiler, which decides at run time, draw byte-identical pictures — and we test that on every demo."

04 The 3D alphabet Stage 3

The 3D turtle carries three directions — **H** (heading), **L** (left) and **U** (up) — the frame used in *The Algorithmic Beauty of Plants*. The frontend's job is to turn symbols into operations:

Symbol	Meaning	Turtle IR
`F`	grow a round branch (tube)	FORWARD
`G`	grow a flat blade (grass, leaf) — in 2D still a line	BLADE
`+ -`	turn left / right (around U)	TURN_LEFT / TURN_RIGHT
`& ^`	pitch down / up (around L)	PITCH_DOWN / PITCH_UP
`\ /`	roll left / right (around H)	ROLL_LEFT / ROLL_RIGHT
`|`	turn around	TURN_LEFT 180
`!`	next parts are thinner (× `taper`)	THIN
`'`	next colour in `colors`	NEXT_COLOR

How it was added.

- **New fields** in the lexer and parser: `dimensions: 3;` `width:` `taper:` `colors: ("#4a2f1c", …);` — each validated (dimensions must be 2 or 3, taper in (0, 1], colours must look like `#rrggbb`).
- `ir.hpp`: seven new operations **appended after** `Pop`, so every Stage 1–2 operation keeps its number and old code is unaffected.
- **One shared function**, `terminalCmd(decl, symbol)` in `lower.hpp`, says what a single symbol draws. The lowering uses it, and so does Member 2's new recursive code generator — so both modes can never disagree about a symbol.
- In a 2D fractal the 3D-only symbols are ignored, so old grammars draw exactly as before.

05 Scenes: many fractals in one file Stage 3

What it is. A file can now hold several fractals and a `scene` that places them:

```
fractal Tree { dimensions: 3; seed: 42; … }
fractal Tuft { dimensions: 3; seed: 7;  … }

scene Meadow {
    place   Tree at (0, 0, 0);
    scatter Tuft 700 within 30 clear 1.2 clumps 22 vary 0.45 seed 11;
    scatter Tuft 350 within 32 clear 0.8 vary 0.35 seed 13;
    scatter Tuft 250 within 46 clear 30 clumps 12 vary 0.4 seed 12;
}
```

How it was added. `parseFile()` reads one or more fractals and an optional scene (`parseSource()` is kept for single-fractal code and old tests). `validateFile()` then checks the whole file: fractal names are unique, the scene only places fractals that exist, and those fractals are 3D. A file with several fractals but no scene is an error.

How placement works (`scene.hpp`, all at compile time):

- **Uniform scatter**: a random point in the ring between `clear` and the radius. The radius uses a square root ($r = \sqrt{(r_0^2 + u \cdot (r_1^2 - r_0^2))}$) so points are spread evenly by *area*, not bunched in the middle.

- **Clumps** (real patches): `clumps N` picks N patch centres; each tuft joins one and lands at a Gaussian distance from it (Box–Muller), and tufts near a centre grow taller — so grass forms mounds like a real lawn.
- **Variety**: every copy gets a random rotation, a size of `1 ± vary`, and **its own seed** (`instanceSeed()`), so 1,300 tufts from one grammar all grow differently.

> **WHY AT COMPILE TIME**
> Positions, sizes and seeds become constants in the compiled program, so every build of the same file gives the same meadow — and the unrolled and recursive backends receive the identical instance list. Defaults (`vary 0.2`, no clumps) reproduce the old placement exactly.

06 Smaller improvements worth knowing

- **BOM accepted** (Stage 2): files saved by Windows Notepad start with three invisible bytes; the lexer now skips them.
- **Branch depth** (Stage 2): `maxBranchDepth()` reports how deeply `[` nests — shown by `--stats`, in the JSON, and in the studio (the plant reaches depth 12).
- `fr_frontend --fractal NAME` (Stage 3): inspect one fractal of a scene file. The JSON export now includes dimensions, seed, width, taper and colours. The studio's live **Stages** tab is built on exactly this command.
- **New error messages**, all with line numbers: weights must be > 0 · seed must be a whole number ≥ 0 · `'dimensions' must be 2 or 3` · `taper must be in (0, 1]` · colour must look like `#4f8a2b` · scene places unknown fractal · several fractals need a scene · `'clear'`, `'clumps'`, `'vary'` ranges · iterations ≤ 60.

07 Numbers to quote

13 turtle operations (was 6, 3 used)	**1,301** instances placed for the meadow	**0.6 ms** lex + parse + check + layout (meadow, in the studio)
79,580 frontend test checks, all passing (Stage 1: 1,904)	**0** changes to the Stage 1–2 meaning of old programs	**12** max branch depth in the fractal plant

08 Questions you may be asked

How can a random grammar give the same result every time?

The randomness is a deterministic function of the seed, the rewrite pass and the choice's position. Same inputs, same numbers — like a hash, not a dice roll.

Why weren't the AST or IR headers broken by all these features?

Stage 1 designed for them: `Production` already had a weight and the rule table already held a list. New IR operations were appended after the old ones, so existing numbers never moved.

Is the grammar still LL(1)?

Yes. Every field and scene item starts with its own keyword (`seed`, `place`, `scatter` …), and alternatives are separated by `|`, so one token of lookahead still decides every branch.

What happens with an unbalanced bracket?

It is rejected while parsing, with the line and character position. It can no longer reach the compiled program.

Why compute the scene layout in the frontend instead of at run time?

It makes builds reproducible and lets both backends work from one identical instance list; the backend then stores it as a constant table.

How do 1,300 tufts look different if they share one grammar?

Each copy gets its own seed, so its random rule choices differ, plus its own position, rotation and size.

FRACTL Stages 2–3 · frontend (Member 1) · lexer.hpp, parser.hpp, ast.hpp, ir.hpp, rewriter.hpp, rng.hpp, lower.hpp, scene.hpp, frontend.hpp, json.hpp, fr_frontend.cpp